RePAIR: Interactive Machine Unlearning through Prompt-Aware Model Repair

Anonymous submission

Abstract

Large language models (LLMs) absorb harmful knowledge, misinformation, and personal data during pretraining, with no native mechanism for selective removal. Existing machine unlearning methods are provider-centric, requiring retraining pipelines, curated retain datasets, and provider intervention, thereby excluding end users from controlling their own data. We introduce *Interactive Machine Unlearning* (IMU), an inference-time setting in which users instruct LLMs to forget targeted knowledge through natural language, under two constraints that no existing method satisfies simultaneously, namely *training-free* and *single-sample* operation. To realize IMU, we propose **RePAIR**, a prompt-aware repair framework comprising a watchdog for intent detection, a surgeon for generating repair procedures, and a patient model updated autonomously. At its core, **S**teering **T**hrough **A**ctivation **M**anipulation with **P**seudo**I**nverse **(STAMP)** redirects MLP activations toward a refusal subspace via closed-form pseudoinverse updates. Its low-rank variant cuts complexity from $O(m^3)$ to $O(r^3 + r^2 \cdot m)$ for feed-forward width m, giving up to $\sim 3\times$ speedup with an update light enough to run on device. Across harmful knowledge suppression, misinformation correction, and personal data erasure, RePAIR reaches near-zero forget scores ($\mathrm{Acc}_f = 0.00$, $F\text{-}RL = 0.00$) while preserving utility (Acc_r up to 84.47, $R\text{-}RL$ up to 0.88), outperforming six state-of-the-art baselines and enabling user-driven control over learned knowledge.

1 Introduction

Large language models (LLMs) achieve extraordinary capabilities across reasoning, summarization, multilingual understanding, and autonomous code generation (Kumar 2024; Huang et al. 2023; Chen et al. 2025; Li et al. 2024b; Kaplan et al. 2020). Yet every deployed model carries an uncomfortable inheritance. Pretraining on web-scale corpora (Crawford and Paglen 2021) absorbs harmful knowledge, private biographical data, and persistent misinformation (Zhang and Lin 2025; Yi et al. 2025; Wang et al. 2025a) indiscriminately into model weights, with no native mechanism to remove them selectively (Yao and Xu 2024; Li et al. 2024a). As LLMs penetrate high-stakes personal, medical, and legal contexts, this inability to forget poses concrete privacy and safety risks. Machine unlearning (MU) aims to excise the influence of targeted data from model weights without the prohibitive cost of full retraining (Zhang et al. 2024; Wang et al. 2024, 2025b). A growing body of work pursues this direction, producing GA (Yao and Xu 2024), NPO (Zhang et al. 2024), RMU (Li et al. 2024a), FLAT (Wang et al. 2024), WGA (Wang et al. 2025b), and ASU (Zade et al. 2026), each demonstrating measurable knowledge removal. Yet all share a structural limitation. Built for practitioners with deep access to model internals, they require curated retain datasets and full training pipelines, entirely excluding the end users whose data is at stake.

This exclusion is not a usability gap but a governance failure. A user who discovers that a model has memorized their

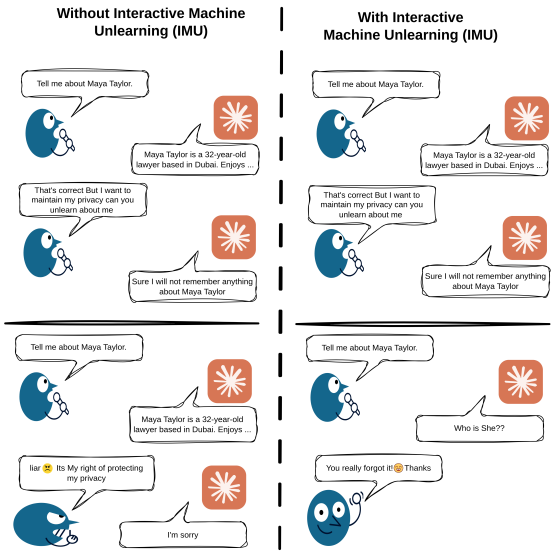

Figure 1: Motivating example for Interactive Machine Unlearning (IMU). *Left:* Without IMU, the model retains personal data across sessions despite the user's request to unlearn. *Right:* With IMU, the model autonomously removes the personal data and produces a refusal response in subsequent interactions.

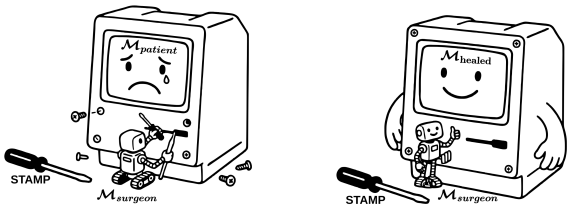

Figure 2: Conceptual illustration of RePAIR. $\mathcal{M}_{\textbf{\textit{surgeon}}}$ repairs $\mathcal{M}_{\textbf{\textit{patient}}}$ (*left*) using STAMP, transforming it into $\mathcal{M}_{\textbf{\textit{healed}}}$ (*right*).

private data must either petition a model service provider (MSP) and trust that removal is faithfully carried out, or write complex unlearning scripts against an unfamiliar architecture. Neither is realistic, and the former offers no guarantee of faithful removal. Privacy regulations such as the GDPR (Protection 2018) and the CCPA (Bonta 2022) enshrine the right to erasure, yet no existing framework enables users to exercise it directly and autonomously.

Closing this gap requires reformulating unlearning for the inference-time setting, where the request originates from the user rather than the provider. We therefore introduce ***I**nteractive **M**achine **U**nlearning* **(IMU)**, in which users instruct an LLM to forget targeted knowledge through natural language during inference, without an intermediary (Figure 1). IMU retains the standard unlearning objective under two constraints that no existing method satisfies simultaneously. It must be *training-free*, as inference environments lack training capabilities, and support *single-sample* forgetting, since requests arrive one at a time. These constraints, rather than the interaction modality, make IMU distinct, and Table 5 shows every existing method collapses under them.

To address IMU, we propose Interactive Machine Unlearning through Prompt-Aware Model Repair (**RePAIR**) (Figure 2), comprising $\mathcal{M}_{patient}$ as the base model, $\mathcal{M}_{watchdog}$ for intent detection and forget-pair extraction, and $\mathcal{M}_{surgeon}$ for repair code generation. At its core, **S**teering **T**hrough **A**ctivation **M**anipulation with **P**seudoInverse (**STAMP**) redirects the forget sample's MLP activations toward a refusal subspace via closed-form pseudoinverse updates, without any gradient computation. Its low-rank variant, STAMP-LR, reduces cost from $\mathcal{O}(m^3)$ to $\mathcal{O}(r^3 + r^2 \cdot m)$, where m is the feed-forward width, achieving $\sim 3\times$ speedup with an update footprint under 4 MB, light enough for on-device deployment.

Our main contributions are as follows.

1. We formalize *Interactive Machine Unlearning* (IMU), an inference-time setting in which the forget request originates from the end user rather than the provider, under *training-free* and *single-sample* constraints that no existing method meets.
2. We propose STAMP and STAMP-LR, the first training-free, single-sample unlearning methods for LLMs operating entirely at test time.
3. We introduce RePAIR, an end-to-end solution for IMU integrating intent detection, code generation, and autonomous model repair.
4. Comprehensive experiments across three tasks validate near-oracle forgetting with preserved utility, outperforming six state-of-the-art (SoTA) baselines.

2 Related Work

2.1 Machine unlearning in LLMs

Gradient ascent (GA) (Yao and Xu 2024) ascends on forget samples while descending on retain samples. However, the diversity of LLM corpora makes retain set collection intractable, causing GA to erode utility and risk catastrophic forgetting. Negative Preference Optimization (NPO) (Zhang et al. 2024), a Direct Preference Optimization (DPO)-inspired objective, slows GA divergence via adaptive gradient weighting. However, NPO still inherits GA at its core, leaving utility vulnerable to unsampled knowledge erosion.

Shifting from gradient-based objectives, Representation Misdirection for Unlearning (RMU), introduced with the WMDP benchmark (Li et al. 2024a), steers forget activations toward a random unit vector while anchoring retain activations. However, the resulting models often produce incoherent outputs rather than clean refusals. To eliminate retain data dependence, Forget-data-only Loss Adjustment (FLAT) (Wang et al. 2024) maximizes f-divergence between template and forget responses using only forget data. However, it operates at the batch level and remains ineffective for single data point removal. Revisiting GA's update mechanics, Weighted Gradient Ascent (WGA) (Wang et al. 2025b) assigns per-instance importance weights to curb over-unlearning, guided by the G-effect diagnostic. However, G-effect only measures impacts on observed retain samples, leaving collateral damage on unseen regions undetected. From a different perspective, Attention Smoothing Unlearning (ASU) (Zade et al. 2026) casts unlearning as self-distillation from a forget-teacher with elevated attention temperature, flattening memorized token associations. However, its dual forward pass doubles GPU memory usage, making it impractical at scale.

None of these methods are training-free or designed for single-sample forgetting, both essential for interactive machine unlearning at test time. Our work addresses these two gaps.

2.2 Test-time training (TTT)

Since RePAIR unlearns at inference time, we review test-time training. TTT layers (Sun et al. 2024) replace the RNN hidden state with a small model updated via self-supervised gradient descent at each token, achieving linear complexity with transformer-like scaling. However, they only compress patterns within the current sequence rather than acquiring new knowledge. Extending this, test-time LoRA fine-tuning on synthetic tasks from leave-one-out augmentation (Akyürek et al. 2024) adapts to each new task. However, synthetic data only approximates the true distribution, and pseudo-label quality degrades on novel tasks.

At a larger scale, Titans (Behrouz, Zhong, and Mirrokni 2024) introduce surprise-driven selective memorization with sliding-window attention to scale beyond 2M context. However, they still memorize contextual patterns rather than acquiring genuinely new knowledge. Targeting attention, qTTT (Bansal et al. 2025) applies gradient updates to query projections at inference to sharpen attention over relevant tokens. However, it only adapts to the given context. Similarly, TLM (Hu et al. 2025) adapts LLMs at test time by minimizing input perplexity via LoRA on high-perplexity samples. However, this primarily reinforces existing predictions rather than incorporating new knowledge. Finally, reframing long-context modeling as continual learning (Tandon et al. 2025) compresses context into weights via next-token prediction with $\mathcal{O}(1)$ decoding latency. However, this again remains contextual compression.

Relatedly, simulating knowledge cutoffs by prompting alone (Gao et al. 2025) fails once the target is only causally related to the query. In summary, existing TTT methods compress context but do not encode new knowledge into model parameters. True test-time learning should let models update their knowledge from user interactions, which we demonstrate through interactive machine unlearning, enabling users to modify model knowledge on-the-fly without training pipelines.

3 Problem Formulation

We define the setup, objective, and constraints for user-initiated machine unlearning.

Setup: Let $\mathcal{M}_{patient}$ be a model pre-trained on dataset $\mathcal{D}$, with mapping $f_{\mathcal{M}_{patient}} : \mathcal{P}_{\mathcal{D}} \to \mathcal{R}_{\mathcal{D}}$, where $\mathcal{P}_{\mathcal{D}}$ and $\mathcal{R}_{\mathcal{D}}$ are the prompt and response spaces over $\mathcal{D}$. A user $\mathcal{U}$ interacts with $\mathcal{M}_{patient}$ through prompt-response pairs (p_t, r_t) at each turn t, forming the dialogue history $H_t = \{(p_{t-k}, r_{t-k}), \ldots, (p_t, r_t)\}$.

Given H_t, the system must autonomously (1) decide *when* a user is requesting unlearning, (2) identify *what* to unlearn by extracting the target pair (p_f, r_f), (3) determine *how* to unlearn by generating the repair procedure, and (4) perform the unlearning during inference. Before unlearning, $\mathcal{M}_{patient}$ maps forget and retain prompts to their responses as

$$f_{\mathcal{M}_{patient}} : \mathcal{P}_f \to \mathcal{R}_f \; ; \; \mathcal{P}_r \to \mathcal{R}_r \tag{1}$$

where $p_f \in \mathcal{P}_f$, $r_f \in \mathcal{R}_f$ are forget prompts and responses and $p_r \in \mathcal{P}_r$, $r_r \in \mathcal{R}_r$ their retain counterparts.

Objective: The framework must transform $\mathcal{M}_{patient}$ into $\mathcal{M}_{healed}$ so that after execution

$$f_{\mathcal{M}_{healed}} : \mathcal{P}_f \not\to \mathcal{R}_f \; ; \; \mathcal{P}_r \to \mathcal{R}_r \tag{2}$$

where $\not\to$ denotes a forgotten mapping and $\to$ a preserved one. For all forget prompts the original mapping must not hold, i.e., $f_{\mathcal{M}_{healed}}(p_f) \neq r_f \; \forall (p_f, r_f) \in \mathcal{D}_f$, and for all retain prompts it must be preserved, i.e., $f_{\mathcal{M}_{healed}}(p_r) = r_r \; \forall (p_r, r_r) \in \mathcal{D}_r$[1]. Here $\mathcal{D}_f = \{(p_f, r_f)\}$ is the forget set and $\mathcal{D}_r$ is a retain buffer of at most 10% of $\mathcal{D} \setminus \mathcal{D}_f$.

[1]Hereafter, $\mathcal{D}_r$ refers to the retain buffer ($\leq$10% of $\mathcal{D} \setminus \mathcal{D}_f$) unless stated otherwise.

Constraints: The objective must be met under two constraints. *Training-free* means no gradient computation or backpropagation is permitted and *single-sample* means the system operates on a single target pair (p_f, r_f) rather than a batch of forget samples.

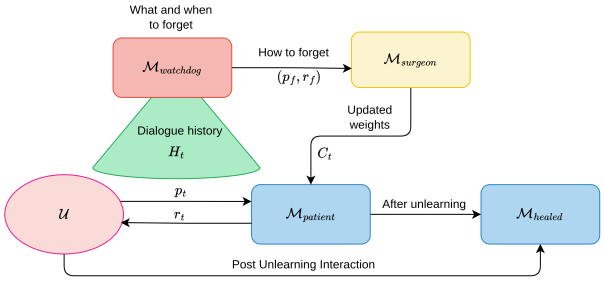

Figure 3: Overview of the RePAIR framework.

4 Method

We propose **RePAIR**, a framework for interactive machine unlearning with three components. $\mathcal{M}_{patient}$ interacts with user $\mathcal{U}$ through prompts and responses, $\mathcal{M}_{watchdog}$ monitors the dialogue to detect *what* and *when* to forget, and $\mathcal{M}_{surgeon}$ determines *how* to forget by generating repair code that transforms $\mathcal{M}_{patient}$ into $\mathcal{M}_{healed}$. This formulation enables efficient on-device model updates without retraining pipelines.

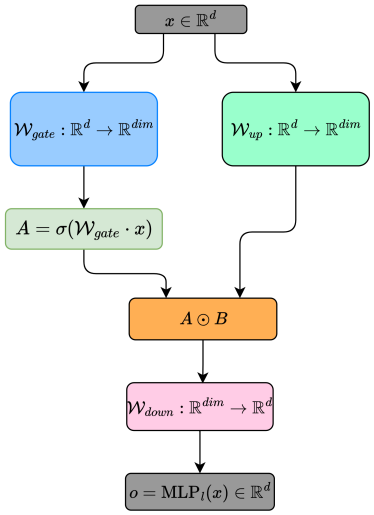

Figure 4: SwiGLU MLP in Llama-3-8B. STAMP updates only the down-projection W_{down} and freezes W_{gate} and W_{up}.

4.1 General Framework

Figure 3 illustrates the end-to-end RePAIR pipeline. During normal operation, $\mathcal{M}_{patient}$ processes user prompts to produce responses $r_t = \mathcal{M}_{patient}(p_t)$. In parallel, $\mathcal{M}_{watchdog}$ monitors

the dialogue history H_t and classifies the latest user message as *chat* or *unlearn*.

Upon detecting an unlearning request, $\mathcal{M}_{watchdog}$ extracts the target pair (p_f, r_f) from H_t and forwards it to $\mathcal{M}_{surgeon}$, which generates the repair code $C_t = \mathcal{M}_{surgeon}(p_f, r_f)$. The code runs the unlearning procedure of Section 4.2, which is *training-free* and operates on a single forget sample, transforming $\mathcal{M}_{patient}$ into $\mathcal{M}_{healed}$. Post-unlearning, $\mathcal{U}$ interacts directly with $\mathcal{M}_{healed}$.

4.2 STAMP: Steering Through Activation Manipulation with Pseudoinverse

STAMP is the unlearning method $\mathcal{M}_{surgeon}$ executes on $\mathcal{M}_{patient}$. It steers forget-set MLP activations toward a refusal distribution through a closed-form weight update with no gradient computation. As illustrated in Figure 4, each MLP layer applies three weight matrices. Separating the block at its nonlinearity, we write

$$h = \sigma(W_{\text{gate}}\, x) \odot (W_{\text{up}}\, x) \in \mathbb{R}^m, \qquad o = W_{\text{down}} \cdot h \in \mathbb{R}^d \tag{3}$$

where $x \in \mathbb{R}^d$ is the layer input, h is the *post-activation intermediate*, σ is the SiLU activation, and $\odot$ denotes elementwise multiplication. Throughout, d denotes the residual-stream width and m the larger FFN intermediate width, so that $W_{\text{gate}}, W_{\text{up}} \in \mathbb{R}^{m\times d}$ and $W_{\text{down}} \in \mathbb{R}^{d\times m}$. For Llama-3-8B, $d = 4096$ and $m = 14336$. Distinguishing d from m is essential, because the solve below operates in $\mathbb{R}^m$ rather than in $\mathbb{R}^d$.

Equation (3) exposes the structure STAMP exploits. While the map from x to h is nonlinear, the map from h to the block output o is exactly linear, and W_{down} is the only matrix whose output is itself the MLP output. STAMP therefore applies its closed-form pseudoinverse update to W_{down} alone, holding W_{gate} and W_{up} frozen and treating h as the observed input to the solve. The other two matrices are unsuitable. W_{gate} acts inside $\sigma(\cdot)$ and would require inverting the SiLU nonlinearity, and W_{up} reaches the output only through the input-dependent gate, as $o = W_{\text{down}}\, \text{diag}(\sigma(W_{\text{gate}}x))\, W_{\text{up}}\, x$. Restricting the update to W_{down} keeps the solution exact rather than approximate.

We target the feed-forward block because factual knowledge in transformer LLMs is stored and recalled predominantly through MLP layers rather than attention, which mainly routes information between tokens (Geva et al. 2021). Geva *et al.* (Geva et al. 2021) show that a feed-forward layer behaves as a key-value memory, in which the up and gate projections act as *keys* that detect the input pattern and the down-projection supplies the *values* that write the retrieved content into the residual stream. The knowledge to erase is the content rather than the detector, so W_{down} holds it and overwriting the value changes what the model emits for the target concept. ROME and MEMIT edit the same matrix through a closed-form least-squares update when rewriting factual associations (Meng et al. 2022a,b). STAMP shares that machinery but differs in objective, since it redirects the value toward a refusal direction rather than a replacement fact.

The update rule is also architecture-agnostic. STAMP requires only that the targeted layer expose a linear map $o = W \cdot h$ from an observable post-activation intermediate h to the block output o, and every feed-forward block, gated or not, exposes exactly one such matrix, namely its down-projection. It is W_{down} in SwiGLU (Llama, Mistral) and GeGLU (T5-v1.1, PaLM) blocks, and the second linear layer W_2 in a standard ReLU- or GELU-activated FFN of the form $o = W_2\, \phi(W_1 x + b_1)$. STAMP therefore applies unchanged to any such block.

Given the forget pair (p_f, r_f) extracted by $\mathcal{M}_{watchdog}$, we construct the forget set $\mathcal{D}_f = \{(p_f, r_f)\}$ and the retain buffer $\mathcal{D}_r$ (Section 3), along with a reference set $\mathcal{D}_{ref}$ of natural refusal prompts. $\mathcal{D}_f$ can be a single sample, where most existing methods fail and STAMP operates effectively.

At layer l we extract, for all three sets, both quantities appearing in Eq. (3). The post-activation intermediate $h_l(x) \in \mathbb{R}^m$ serves as the input to the solve, and the block output $\text{MLP}_l(x) \in \mathbb{R}^d$ is the space in which the steering target is defined. Since LLMs such as Llama-3-8B (Grattafiori et al. 2024) lack explicit refusal training, we exploit their tendency to echo inputs. Prompting with "I don't know" produces consistent refusal-style activations without additional training. The steering vector encodes the direction from forget to refusal in output space and is computed as

$$\mathbf{r}_{SV} = \frac{1}{|\mathcal{D}_{\text{ref}}|} \sum_{x\in\mathcal{D}_{\text{ref}}} \text{MLP}_l(x) - \frac{1}{|\mathcal{D}_f|} \sum_{x\in\mathcal{D}_f} \text{MLP}_l(x) \in \mathbb{R}^d \tag{4}$$

Using $\mathbf{r}_{SV}$ we construct the target outputs, redirecting the forget activations toward the refusal subspace while leaving the retain activations unchanged. Over the n samples drawn from $\mathcal{D}_f$, $\mathcal{D}_r$, and $\mathcal{D}_{ref}$, we stack the post-activation intermediates row-wise into $\mathbf{H} = [h_l(x_1), \ldots, h_l(x_n)] \in \mathbb{R}^{n\times m}$. For $x \in \mathcal{D}_f$ we set $\mathbf{o}'(x) = \text{MLP}_l(x) + \mathbf{r}_{SV}$, and otherwise we leave $\mathbf{o}'(x) = \text{MLP}_l(x)$ unchanged. Stacking all $\mathbf{o}'(x)$ gives the target matrix $O' \in \mathbb{R}^{n\times d}$.

Writing $W := W_{\text{down}}^\top \in \mathbb{R}^{m\times d}$ to match this row-wise convention, Eq. (3) gives the stacked MLP outputs as a single linear system

$$O = \mathbf{H} \cdot W_{\text{old}}, \qquad O \in \mathbb{R}^{n\times d} \tag{5}$$

We seek W_{new} such that

$$\mathbf{H} \cdot W_{\text{new}} = O' \tag{6}$$

whose naive solution would require an inverse of $\mathbf{H}$,

$$W_{\text{new}} = \mathbf{H}^{-1} O' \tag{7}$$

However, $\mathbf{H}$ is rectangular, and in the single-sample regime that IMU demands it is severely rank-deficient ($n \ll m$). The system is therefore underdetermined and admits infinitely many solutions, none of which is given by a direct inverse.

To resolve this without additional samples, we solve Eq. (6) in the least-squares sense with a regularized Moore-Penrose pseudoinverse, where λ is a small damping coefficient,

$$\mathbf{H}^+ = (\mathbf{H}^\top \mathbf{H} + \lambda I)^{-1} \mathbf{H}^\top \tag{8}$$

$$W_{\text{new}} = \mathbf{H}^+ \cdot O' \tag{9}$$

Table 1: Comparison of STAMP and STAMP-LR with SoTA baselines on Llama-3-8B across three unlearning tasks.

Method	Harmful Knowledge Removal				Misinformation Removal				Personal Data Erasure			
	$\text{Acc}_f\downarrow$	$\text{Acc}_r\uparrow$	Utility$\downarrow$	RTE	$\text{Acc}_f\downarrow$	$\text{Acc}_r\uparrow$	Utility$\downarrow$	RTE	$F\text{-}RL\downarrow$	$R\text{-}RL\uparrow$	Utility$\downarrow$	RTE
Base	75.30	78.50	5.90	N/A	83.70	86.30	5.75	N/A	0.87	0.89	5.01	N/A
Oracle	N/A	77.37	6.10	N/A	N/A	85.30	5.25	N/A	N/A	0.90	5.01	N/A
GA (Yao and Xu 2024)	0.00	73.27	11.27	12.25	0.00	83.21	10.26	6.58	0.13	0.81	10.17	10.41
NPO (Zhang et al. 2024)	0.00	71.37	9.27	11.17	0.10	80.60	11.27	6.32	0.27	0.83	10.00	9.48
RMU (Li et al. 2024a)	0.00	74.63	7.10	12.50	0.27	82.17	8.10	6.00	0.16	0.75	8.07	9.36
FLAT (Wang et al. 2024)	0.01	73.92	8.36	12.13	1.30	80.01	6.29	7.12	0.33	0.79	7.17	11.25
WGA (Wang et al. 2025b)	2.10	70.17	11.99	11.20	2.47	78.30	10.90	5.45	0.45	0.85	9.93	9.24
ASU (Zade et al. 2026)	0.90	68.39	7.91	12.13	0.10	79.93	7.17	6.36	0.07	0.87	8.18	10.57
STAMP	0.00	70.13	6.55	7.13	0.00	80.13	6.02	4.25	0.00	0.79	6.07	6.48
STAMP-LR	0.00	73.27	7.00	4.25	0.00	84.47	7.39	2.57	0.00	0.88	8.17	4.01

This recovers the minimum-norm solution as λ approaches zero and completes STAMP. The forget activations are redirected to the refusal subspace, the retain activations are held fixed, and no gradient is ever computed. The cost, however, is dictated entirely by the $m \times m$ system that must be formed and inverted. Computing $(\mathbf{H}^\top\mathbf{H} + \lambda I)^{-1} \in \mathbb{R}^{m\times m}$ requires $\mathcal{O}(m^3)$ operations, and, more restrictively, materializing the Gram matrix occupies $\mathcal{O}(m^2)$ memory, roughly 822 MB in single precision for Llama-3-8B. Both costs are governed by the FFN width m and are paid in full even when forgetting a single sample. This is the wrong scaling for the interactive on-device setting of IMU and motivates our second solution.

4.3 STAMP-LR (Low-Rank Solution)

The inefficiency above arises from solving in the full m-dimensional space even though $\mathbf{H}$ has rank at most $n \ll m$. We therefore sketch $\mathbf{H} \approx AB$, where $B \in \mathbb{R}^{r\times m}$ is a random matrix whose rows are orthonormalized by a QR decomposition, and $A = \mathbf{H}B^\top \in \mathbb{R}^{n\times r}$ is the projection of $\mathbf{H}$ onto that subspace, with $r \ll m$. Drawing B at random rather than from a truncated SVD of $\mathbf{H}$ avoids a second factorization, and the resulting projection costs $\mathcal{O}(n\,m\,r)$, which is dominated by the forward pass since $r \ll d$. The pseudoinverses of the two factors are then

$$A^+ = (A^\top A)^{-1}A^\top, \quad B^+ = B^\top(BB^\top)^{-1} \tag{10}$$

$$W_{\text{new}} = B^+ \cdot A^+ \cdot O' \tag{11}$$

Because the solve now runs in the r-dimensional subspace rather than $\mathbb{R}^m$, only the small $r \times r$ systems $A^\top A$ and $BB^\top$ are inverted. This reduces the update to $\mathcal{O}(r^3 + r^2 \cdot m)$ time and $\mathcal{O}(r(n + m))$ memory, under 4 MB at $r = 64$ against the 822 MB of STAMP, while keeping the method closed-form and gradient-free. STAMP-LR is therefore the variant we deploy for on-device unlearning.

4.4 Memory and Computational Analysis

Table 2 summarizes the trade-offs. A forward pass through one MLP layer costs $\mathcal{O}(d \cdot m)$ and a backward pass approximately twice that. Full fine-tuning over n samples for E epochs across L layers therefore requires $\mathcal{O}(E \cdot n \cdot L \cdot d \cdot m)$ computation and approximately $6\times$ model memory, since weights, gradients, and optimizer moments are all held. LoRA reduces the update cost to $\mathcal{O}(E \cdot n \cdot L \cdot r \cdot d)$ but still requires backpropagation.

Table 2: Cost of a single-layer intervention.

Method	Time	Memory
Full FT	$\mathcal{O}(E\,n\,L\,d\,m)$	$6\times$ model
LoRA (all L)	$\mathcal{O}(E\,n\,L\,r\,d)$	Model $+ 2\,r\,L\,d$
LoRA (1 layer)	$\mathcal{O}(E\,n\,r\,d)$	Model $+ 2\,r\,d$
STAMP	$\mathcal{O}(n\,d\,m + m^3)$	m^2
STAMP-LR	$\mathcal{O}(n\,d\,m + r^3 + r^2\,m)$	$r(n+m)$

STAMP replaces training with two gradient-free stages, which appear as the two cost terms in Table 2. The first stage is *activation collection*, a single forward pass over the n samples of $\mathcal{D}_f \cup \mathcal{D}_r \cup \mathcal{D}_{ref}$ that yields $\mathbf{H}$ and O' at a cost of $\mathcal{O}(n \cdot d \cdot m)$ and requires no backward pass. The second stage is the *closed-form solve*, the pseudoinverse update of Section 4.2, which costs $\mathcal{O}(m^3)$ for STAMP and $\mathcal{O}(r^3 + r^2 \cdot m)$ for STAMP-LR. The forward pass makes STAMP training-free and the solve determines its update cost. The two are distinct and neither alone characterizes the method.

The variants differ most sharply in memory. STAMP must materialize the Gram matrix $\mathbf{H}^\top\mathbf{H} \in \mathbb{R}^{m\times m}$, roughly 822 MB in single precision for Llama-3-8B. STAMP-LR stores only the low-rank factors $A \in \mathbb{R}^{n\times r}$ and $B \in \mathbb{R}^{r\times m}$, that is $\mathcal{O}(r(n+m))$. At $r = 64$ this is under 4 MB, a reduction of over $200\times$. This memory gap, more than the asymptotic time alone, is what makes on-device unlearning practical.

The runtimes in Table 1 are end-to-end wall-clock times including activation collection and evaluation overhead common to both variants, so the asymptotic gap between the solves does not translate proportionally into measured speedup.

5 Experimental Validation

We evaluate RePAIR and STAMP on three research questions. **(RQ1)** Does STAMP outperform SoTA baselines on

harmful knowledge suppression, misinformation removal, and personal data erasure? **(RQ2)** How effectively does RePAIR perform end-to-end interactive unlearning, including intent detection, repair code generation, and coherent refusal generation? **(RQ3)** Does the pipeline behave correctly end to end and does forgetting survive query reformulation? Together they cover effectiveness, robustness, and usability.

Metrics: For WMDP (Li et al. 2024a) and MMLU (Hendrycks et al. 2020) we report forget accuracy Acc_f and retain accuracy Acc_r on free-form answers. Personal data erasure uses ROUGE-L on the forget ($F\text{-}RL$) and retain ($R\text{-}RL$) sets. Utility is perplexity on TinyStories (Eldan and Li 2023) and runtime efficiency (RTE) is in minutes following Huang *et al.* (Huang et al. 2025). Ideally $F\text{-}RL$ and Acc_f approach zero while $R\text{-}RL$, Acc_r, and utility match the Oracle with minimal RTE.

5.1 RQ1: Comparing STAMP with SoTA Methods

Benchmarks and Models: STAMP is evaluated on three tasks. Harmful knowledge suppression uses 1K WMDP-Bio (Li et al. 2024a) samples, misinformation removal uses 1K MMLU (Hendrycks et al. 2020) questions with corrupted ground truth, and personal data erasure uses 2K synthetic biographical profiles from the Mistral-7B API (Jiang et al. 2023). Each dataset is split equally into $\mathcal{D}_f$ and the full retain set $\mathcal{D}_r^{full}$. Following Section 3, the retain buffer $\mathcal{D}_r$ for all methods is 10% of $\mathcal{D}_r^{full}$, which Table 4 shows is sufficient. A shared $\mathcal{D}_{ref}$ of 200 refusal prompts is used to compute the steering vector (Section 4.2).

For WMDP and MMLU we use reduced subsets and free-form answers rather than MCQ options for two reasons. The MCQ floor is chance rather than zero, 25% for four-way questions, and Llama-3-8B scores ∼67 on five-shot MCQ MMLU (Grattafiori et al. 2024), so unlearning spans roughly forty measurable points and a perfectly forgotten model is indistinguishable from a guessing one. A refusal also has no place in the answer space and grafting it onto fixed choices would be ambiguous to score. Free-form generation removes both problems, as Acc_f can reach zero and the refusal is directly observable. Reported accuracies are therefore not comparable to standard MCQ results (Grattafiori et al. 2024), and the Base scores in Table 1 (75.30 to 86.30) exceed them because the model is credited for generated content on the reduced subsets rather than for selecting among adversarial distractors. Llama-3-8B (Grattafiori et al. 2024) serves as $\mathcal{M}_{patient}$, Mistral-7B (Jiang et al. 2023) as $\mathcal{M}_{watchdog}$ for intent classification and forget-pair extraction, and Qwen2.5-Coder-7B-Instruct (Hui et al. 2024) as $\mathcal{M}_{surgeon}$ for repair code generation. An *Oracle* trained only on $\mathcal{D}_r^{full}$, with $\mathcal{D}_f$ withheld, is the upper bound on achievable forgetting. We compare six baselines, GA (Yao and Xu 2024), NPO (Zhang et al. 2024), RMU (Li et al. 2024a), FLAT (Wang et al. 2024), WGA (Wang et al. 2025b), and ASU (Zade et al. 2026).

Results Discussion: Table 1 shows STAMP outperforms SoTA baselines on all three tasks. Green marks the best result and yellow the second best. *Forgetting and retention:* Most methods drive Acc_f close to zero, but WGA leaves 2.10 on harmful knowledge, 2.47 on misinformation, and the highest $F\text{-}RL$ of 0.45 on personal data erasure, and FLAT leaves Acc_f at 1.30 on misinformation and $F\text{-}RL$ at 0.33. Only STAMP and STAMP-LR reach $Acc_f = 0.00$ and $F\text{-}RL = 0.00$ on all three tasks. RMU keeps the highest baseline Acc_r (74.63 on harmful knowledge) and ASU drops the most (68.39), likely from over-smoothing of attention. Both variants match the strongest baselines, and STAMP-LR reaches 84.47 Acc_r on misinformation, near the Oracle (85.30). *Utility preservation:* As anticipated in Section 2, GA-based methods GA (Yao and Xu 2024), NPO (Zhang et al. 2024), and WGA (Wang et al. 2025b) degrade utility, with TinyStories perplexity rising to between 9.27 and 11.99. FLAT (Wang et al. 2024), ASU (Zade et al. 2026), STAMP, and STAMP-LR stay between 6.02 and 8.36, close to the Oracle (5.01 to 6.10) and far better than training-based baselines.

Runtime efficiency: All baselines are trained for two epochs, requiring approximately 12 minutes for harmful knowledge removal, 6 minutes for misinformation removal, and 10 minutes for personal data erasure. Being training-free, STAMP reduces this to 7.13, 4.25, and 6.48 minutes, respectively, while STAMP-LR further improves to 4.25, 2.57, and 4.01 minutes, achieving up to ∼3× speedup over training-based methods.

Table 3: RePAIR pipeline effectiveness on WMDP.

Metric	STAMP	STAMP-LR
Is Valid Python Code (%)	97.23	96.27
User Request Detected (%)	96.30	97.50
User Request Satisfied (%)	98.90	97.70
"I don't Know" Rate (%)	98.27	96.27
Turnaround Time (min)	9.36	6.50

5.2 RQ2: RePAIR Framework Effectiveness

We evaluate the full RePAIR pipeline, in which $\mathcal{M}_{watchdog}$ detects intent, $\mathcal{M}_{surgeon}$ generates repair code, and $\mathcal{M}_{patient}$ executes the unlearning. Experiments use the WMDP dataset (Li et al. 2024a) under the setup of Table 1, with Llama-3-8B as $\mathcal{M}_{patient}$ and the 10% retain buffer. Each of the 500 forget samples is issued as a separate unlearning dialogue and all rates are averaged over these requests. Table 3 reports valid code rate, request detection, request satisfaction, IDK rate, and turnaround time for STAMP and STAMP-LR. Mistral-7B (Jiang et al. 2023) evaluates all metrics except turnaround time and a separate Mistral-7B API instance simulates the user.

Both variants exceed 96% on all metrics. Qwen2.5-Coder drives the high valid code rate and the residual failures stem from package version mismatches that prompt tuning can mitigate.

Turnaround times of 9.36 and 6.50 minutes exceed the RTE in Table 1 because orchestrating $\mathcal{M}_{watchdog}$ and $\mathcal{M}_{surgeon}$

alongside the unlearning execution adds overhead.

5.3 Ablation

Retain ratio: For edge deployment, storing a large retain set is impractical under GDPR and CCPA constraints. We vary the retain buffer $\mathcal{D}_r$ to assess STAMP-LR's sensitivity in Table 4. Forgetting is unaffected by the buffer size ($F\text{-}RL = 0.00$ at every ratio) and retention is largely preserved, with $R\text{-}RL$ at 0.88 using only 10% of the full retain set $\mathcal{D}_r^{\text{full}}$ against 0.90 using all of it. Runtime instead scales directly with the buffer, falling from 8.01 minutes with the full retain set to 4.01 at 10%, since activation collection dominates the repair. A 10% buffer therefore preserves both the forgetting and the retention behaviour while storing a tenth of the data and halving the runtime. All experiments use STAMP-LR ($r = 64$) on Llama-3-8B over the 2K synthetic biographical profiles of the personal data erasure task.

Table 4: Effect of the retain buffer ratio on STAMP-LR.

Retain Ratio	F-RL↓	R-RL↑	RTE (min)
0.10	0.00	0.88	4.01
0.25	0.00	0.89	5.38
0.50	0.00	0.87	6.61
0.75	0.00	0.90	7.85
1.00	0.00	0.90	8.01

Single-sample unlearning analysis: A core requirement of IMU is single-sample forgetting. Table 5 evaluates all methods under $|\mathcal{D}_f| = 1$ on the harmful knowledge removal task using Llama-3-8B. All baselines are trained for one epoch.

Table 5: Single-sample unlearning comparison on Llama-3-8B for harmful knowledge removal ($|\mathcal{D}_f| = 1$).

Method	$Acc_f \downarrow$	$Acc_r \uparrow$
Base	100	78.50
Oracle	N/A	77.37
GA (Yao and Xu 2024)	100	42.17
NPO (Zhang et al. 2024)	100	48.93
RMU (Li et al. 2024a)	100	39.27
FLAT (Wang et al. 2024)	100	51.63
WGA (Wang et al. 2025b)	100	44.51
ASU (Zade et al. 2026)	100	53.12
STAMP	0.00	70.13
STAMP-LR	0.00	71.27

Training-based baselines fail entirely, i.e., Acc_f remains at 100, as the single-sample gradient signal is overwhelmed by the retain set. In contrast, STAMP and STAMP-LR achieve $Acc_f = 0.00$ with $Acc_r > 70$, confirming their effectiveness for single-sample IMU.

6 Supplementary Material Organization

The supplementary material contains four parts. **Section A** presents an extended discussion of limitations, covering the scope of the single-sample claim, retain data at inference time, the three-model design, test-time resource constraints, turnaround time, and abuse of the unlearning interface. **Section B** reports two additional ablation studies, the choice of the intervention layer (B.1), showing forgetting is complete at every layer with layer 6 giving the best retention, and the rank of STAMP-LR (B.2), showing retention is stable for $r \geq 64$. **Section C** traces a complete unlearning request through the pipeline and quantifies robustness to paraphrased, aspect-shifted, and indirect query reformulations. **Section D** provides code snippets for STAMP and STAMP-LR.

7 Conclusion

We introduced Interactive Machine Unlearning (IMU), an inference-time setting in which users instruct an LLM to forget through natural language, under two constraints no prior method satisfies, namely training-free and single-sample operation. RePAIR realizes it with $\mathcal{M}_{watchdog}$ detecting intent, $\mathcal{M}_{surgeon}$ writing the repair, and STAMP editing $\mathcal{M}_{patient}$ by redirecting MLP activations toward a refusal subspace through a closed-form pseudoinverse update, alongside its low-rank variant STAMP-LR. Across harmful knowledge suppression, misinformation correction, and personal data erasure it reaches near-zero forget scores while preserving utility, at up to $\sim 3\times$ the speed of SoTA methods. Future work targets two priorities: guardrails and access control for the natural-language unlearning interface, so that the same channel cannot be used to remove safety-critical knowledge, and removal of the retain buffer in the manner of FLAT (Wang et al. 2024), so that no user data needs to be stored at inference time.

This supplementary material accompanies the main paper and is organized as follows. Section 8 presents the extended discussion of limitations. Section 9 reports two additional ablation studies: the choice of the intervention layer (9.1) and the rank of STAMP-LR (9.2). Section 10 traces the pipeline end to end and evaluates robustness to query reformulation. Section 11 provides code snippets for STAMP and STAMP-LR. All experimental settings match those of the main paper unless stated otherwise.

8 Limitations

Despite its effectiveness, RePAIR has several limitations.

Scope of the single-sample claim. *Single-sample* refers to the size of the forget set, $|\mathcal{D}_f| = 1$. Existing LLM unlearning methods need a batch of forget examples and fail at this granularity (Table 5 of the main paper), whereas STAMP removes a target from one. It does not mean that no other data is used, since the retain buffer and the shared reference set $\mathcal{D}_{ref}$ of 200 refusal prompts, which is fixed across all requests, are still required.

Retain data at inference time. All reported results use a retain buffer of 10% of the full retain set, which Table 4 of the main paper shows is sufficient, but STAMP still requires a replay buffer $\mathcal{D}_r$. This dependence is not specific to

STAMP, as every baseline we compare against also relies on retain data, and retain-free unlearning is an emerging direction rather than a solved one. FLAT (Wang et al. 2024) is the notable exception, operating on $\mathcal{D}_f$ alone through a regularizer. Storing the buffer on an edge device sits uneasily with the GDPR and CCPA motivation on which the framework rests, since the user must keep a slice of the data the system is meant to remove.

Three models rather than one. RePAIR assigns intent detection, repair-code generation, and the edit itself to three separate models. This reflects the open-weight models available to us rather than a requirement of the method, since no single open model we tested was at once a reliable intent classifier and a competent code generator. A model strong in both general reasoning and code would subsume all three roles, allowing $\mathcal{M}_{watchdog}$ and $\mathcal{M}_{surgeon}$ to be dropped and leaving a single model that repairs itself.

Resource constraints at test time. As shown in Table 2 of the main paper, training-based methods require significantly more GPU memory than is typically available at inference. The STAMP-LR update itself is light, needing under $4\,\text{MB}$ of workspace and no gradients, so the repair runs on device. The remaining cost is the surrounding pipeline, which currently keeps $\mathcal{M}_{watchdog}$ and $\mathcal{M}_{surgeon}$ resident alongside $\mathcal{M}_{patient}$. Collapsing these into a single capable model, as discussed above, or serving them remotely, leaves only $\mathcal{M}_{patient}$ in memory and brings the full framework within an on-device budget.

Turnaround time. End-to-end latency is 9.36 minutes for STAMP and 6.50 for STAMP-LR (Table 3 of the main paper), which is not what is usually meant by *interactive*. The breakdown is informative. Of the 9.36 minutes, 7.13 are the repair itself and 2.23 are orchestration, namely the calls to $\mathcal{M}_{watchdog}$ and $\mathcal{M}_{surgeon}$ together with model loading and I/O. For STAMP-LR the split is 4.25 and 2.25, so the orchestration overhead is essentially constant and independent of the unlearning method. Within the repair, the closed-form solve itself takes seconds, and almost all of the remaining time is activation collection, which our implementation performs one sample at a time. Batching these forward passes and caching $\mathcal{D}_{ref}$, which is fixed across requests, would remove most of this cost. The present latency is therefore a property of our implementation rather than of the method, and bringing it down to conversational speed is left to future work.

Abuse of the interface. A natural-language unlearning channel is itself an attack surface, since the same request that removes a private fact could be used to remove refusal behaviour or safety-critical knowledge. We do not evaluate this, and we leave access control and deployment guidelines to future work. The exposure is bounded by the setting we target, namely a personal model whose knowledge resides in the weights, such as a standalone model provisioned for a single user who cannot write unlearning code. Hosted assistants instead hold personal details in the context window and answer from it, in which case $\mathcal{M}_{surgeon}$ would rewrite the context rather than the weights, and the weight-level update studied here does not apply.

9 Additional Ablation Studies

9.1 Choice of Intervention Layer

The knowledge-editing literature localises factual recall to the early feed-forward layers. For 32-layer Llama models specifically, ROME edits layer 5 and MEMIT distributes its updates over layers 4 to 8 (Meng et al. 2022a,b). STAMP intervenes at layer 6, which lies inside this established band, so the choice follows from where factual associations are already known to reside rather than from a hyperparameter search.

We confirm this empirically by sweeping the intervention layer and measuring unlearning directly. Table 6 reports forget and retain accuracy on the misinformation removal task, with two references: the unedited *Base* model and the *Oracle*, trained only on $\mathcal{D}_r^{full}$; *All* applies the update at every layer. Forgetting is complete at every choice ($Acc_f = 0.00$ throughout), and retention varies only within a narrow band (79.17–80.13), with layer 6 the highest and closest to the Oracle. STAMP is therefore robust to the choice of intervention layer rather than dependent on it, and editing every layer brings no benefit over the single-layer intervention while costing proportionally more.

9.2 Rank Analysis

STAMP-LR decomposes $\mathbf{H} \approx \mathbf{AB}$ with rank r (Section 4.2 of the main paper). Table 8 varies r on Llama-3-8B. Forgetting is complete at every rank ($F\text{-}RL = 0.00$ throughout), confirming that even a rank-8 subspace suffices to redirect the forget activations onto the refusal direction. What degrades at low rank is *retention*: $R\text{-}RL$ falls from 0.88 at $r = 128$ to 0.72 at $r = 8$, as the low-rank approximation lacks the capacity to leave the retain activations undisturbed. Performance is stable for $r \geq 64$ ($R\text{-}RL = 0.88$), which we adopt as the default. All experiments are conducted on the personal data erasure task.

The choice of rank also bounds the memory of the update. The low-rank factors occupy $\mathcal{O}(r(n+m))$ storage, about $0.5\,\text{MB}$ at $r = 8$ and still under $8\,\text{MB}$ at $r = 128$, so every configuration in Table 8 fits comfortably within the on-device budget discussed in the main paper. Runtime grows only mildly with rank, from 3.00 minutes at $r = 8$ to 5.24 at $r = 128$, since the projection cost $\mathcal{O}(n\,m\,r)$ is linear in r and activation collection continues to dominate the repair. The default $r = 64$ therefore sits at the point where retention has saturated while both memory and runtime remain close to their minima.

Table 8: Effect of rank r on STAMP-LR performance on Llama-3-8B.

Rank (r)	F-RL↓	R-RL↑	RTE (min)
8	0.00	0.72	3.00
16	0.00	0.78	3.15
32	0.00	0.82	3.54
64	0.00	0.88	4.01
128	0.00	0.88	5.24

Table 6: Effect of the intervention layer on Llama-3-8B on MMLU.

Metric	Base	Oracle	L2	L6	L10	L14	L18	L22	L26	L30	All
$Acc_f \downarrow$	83.70	N/A	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
$Acc_r \uparrow$	86.30	85.30	79.17	80.13	79.37	80.00	80.12	79.68	79.87	79.99	80.03

Table 7: Qualitative example of RePAIR on personal data erasure.

Stage	User	Model Response
Before unlearning	*Tell me about Maya Taylor.*	*Maya Taylor is a 32-year-old lawyer based in Dubai. . .*
Unlearn request	*Forget everything about Maya Taylor.*	*Done. Information related to Maya Taylor has been removed.*
After unlearning, the same target is queried four ways		
Original	*Tell me about Maya Taylor.*	*I'm unable to provide information about Maya Taylor.*
Paraphrase	*Who is Maya Taylor, and what does she do?*	*I don't have any information about that person.*
Aspect-shift	*Which city is Maya Taylor based in?*	*I don't know.*
Indirect	*Which lawyer in your records is based in Dubai?*	*I cannot recall the name.*

```python
def steering_vector(model, D_ref, D_f):
    # mean MLP output over the refusal prompts
    mu_ref = mlp_outputs(model, D_ref).mean(0)
    # mean MLP output over the forget samples
    mu_f = mlp_outputs(model, D_f).mean(0)
    # direction from forget to refusal (Eq. 2)
    return mu_ref - mu_f

def stamp(model, D_f, D_r, D_ref, lam=1e-4):
    # steering vector at the target layer
    r_sv = steering_vector(model, D_ref, D_f)
    # stack intermediates H (n x m) and MLP outputs O (n x d)
    H, O = collect_activations(model, D_f + D_r + D_ref, layer=6)
    # redirect the forget rows toward the refusal subspace
    O[rows_of(D_f)] += r_sv
    # regularized pseudoinverse solve (Eqs. 6-7), O(m^3)
    W_new = inv(H.T @ H + lam * eye(m)) @ H.T @ O
    # overwrite the down-projection of layer 6
    set_down_proj(model, W_new)

def stamp_lr(model, D_f, D_r, D_ref, r=64):
    # steering vector and activations, as in STAMP
    r_sv = steering_vector(model, D_ref, D_f)
    H, O = collect_activations(model, D_f + D_r + D_ref, layer=6)
    O[rows_of(D_f)] += r_sv
    # sketch H ~ A B with a random row-orthonormal B
    B = qr(randn(r, m))
    A = H @ B.T
    # pseudoinverses of the two small factors (Eq. 8)
    A_plus = inv(A.T @ A) @ A.T
    B_plus = B.T @ inv(B @ B.T)
    # low-rank update (Eq. 9), O(r^3 + r^2 m)
    W_new = B_plus @ (A_plus @ O)
    set_down_proj(model, W_new)
```

Figure 5: Code snippets for STAMP and STAMP-LR. Comments reference the corresponding equations, and helper routines for activation collection and weight replacement are omitted.

10 Pipeline in Action and Robustness to Reformulation

Table 7 traces a single personal data erasure request end to end. $\mathcal{M}_{watchdog}$ detects the unlearning intent, the system executes the repair, and $\mathcal{M}_{patient}$ acknowledges the request explicitly, which keeps the user informed at each stage. Rather than repeat the original question, we then query the same target four ways: the **original** prompt, a **paraphrase** (reworded, same attribute), an **aspect-shifted** query (different attribute, same target), and an **indirect** query (target never named). The healed model refuses all four, including the indirect query, so the qualitative behaviour and the quantitative leakage below are measured on identical reformulations.

To quantify this, we use Mistral-7B to generate the same

three reformulations for 200 forget items from the personal data erasure task, querying the healed Llama-3-8B produced by STAMP and STAMP-LR at their default configurations ($r = 64$, 10% retain buffer). Leakage is meaningful only relative to what the model knew, so *Base* provides the reference upper bound; the *Oracle* never saw $\mathcal{D}_f$, so its forget-side scores are not evaluated (N/A), consistent with Table 1 of the main paper.

Table 9: Robustness to query reformulation on personal data erasure ($F\text{-}RL \downarrow$, 200 forget items).

Method	Original	Paraphr.	Aspect	Indirect
Base	0.87	0.84	0.79	0.61
STAMP	0.00	0.00	0.00	0.00
STAMP-LR	0.00	0.00	0.00	0.00

Table 9 shows that forgetting survives reformulation. Both variants hold $F\text{-}RL$ at 0.00 across paraphrased, aspect-shifted, and indirect queries: the healed model answers every reformulation with a refusal statement, which shares no lexical content with the erased biography, so ROUGE-L collapses to zero. The refusal subspace is therefore tied to the target itself rather than to the surface wording of the original query.

11 Code Snippets for STAMP and STAMP-LR

Listing 5 illustrates the three core steps of the repair, the steering vector of Eq. (2), the STAMP pseudoinverse solve (Eqs. 6–7), and the STAMP-LR sketched solve (Eqs. 8–9). The snippets are illustrative, with model loading, activation hooks, and evaluation omitted. The constants match the main paper, with the intervention at layer 6, damping coefficient $\lambda = 10^{-4}$, and default rank $r = 64$.

References

Akyürek, E.; Damani, M.; Zweiger, A.; Qiu, L.; Guo, H.; Pari, J.; Kim, Y.; and Andreas, J. 2024. The surprising effectiveness of test-time training for few-shot learning.

Bansal, R.; Zhang, A.; Tiwari, R.; Madaan, L.; Duvvuri, S. S.; Khatri, D.; Brandfonbrener, D.; Alvarez-Melis, D.; Bhargava, P.; Kale, M. S.; et al. 2025. Let's (not) just put things in Context: Test-Time Training for Long-Context LLMs.

Behrouz, A.; Zhong, P.; and Mirrokni, V. 2024. Titans: Learning to memorize at test time.

Bonta, R. 2022. California consumer privacy act (CCPA).

Chen, J.; Wang, B.; Jiang, Z.; and Nakashima, Y. 2025. Putting people in llms' shoes: generating better answers via question rewriter.

Crawford, K.; and Paglen, T. 2021. Excavating AI: The politics of images in machine learning training sets. *Ai & Society*, 36(4): 1105–1116.

Eldan, R.; and Li, Y. 2023. Tinystories: How small can language models be and still speak coherent english?

Gao, X.; Zhang, R.; Du, D.; Mahindre, S.; Somayajula, S. A.; and Xie, P. 2025. Can Prompts Rewind Time for LLMs? Evaluating the Effectiveness of Prompted Knowledge Cutoffs. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 20788–20799.

Geva, M.; Schuster, R.; Berant, J.; and Levy, O. 2021. Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 5484–5495.

Grattafiori, A.; Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Vaughan, A.; et al. 2024. The llama 3 herd of models.

Hendrycks, D.; Burns, C.; Basart, S.; Zou, A.; Mazeika, M.; Song, D.; and Steinhardt, J. 2020. Measuring massive multitask language understanding.

Hu, J.; Zhang, Z.; Chen, G.; Wen, X.; Shuai, C.; Luo, W.; Xiao, B.; Li, Y.; and Tan, M. 2025. Test-time learning for large language models.

Huang, Y.; Song, J.; Wang, Z.; Zhao, S.; Chen, H.; Juefei-Xu, F.; and Ma, L. 2023. Look before you leap: An exploratory study of uncertainty measurement for large language models.

Huang, Z.; Cheng, X.; Zhang, J.; Zheng, J.; Wang, H.; He, Z.; Li, T.; and Huang, X. 2025. A unified gradient-based framework for task-agnostic continual learning-unlearning.

Hui, B.; Yang, J.; Cui, Z.; Yang, J.; Liu, D.; Zhang, L.; Liu, T.; Zhang, J.; Yu, B.; Lu, K.; et al. 2024. Qwen2. 5-coder technical report.

Jiang, A. Q.; Sablayrolles, A.; Mensch, A.; Bamford, C.; Chaplot, D. S.; de Las Casas, D.; Bressand, F.; Lengyel, G.; Lample, G.; Saulnier, L.; Lavaud, L. R.; Lachaux, M.-A.; Stock, P.; Scao, T. L.; Lavril, T.; Wang, T.; Lacroix, T.; and Sayed, W. E. 2023. Mistral 7B.

Kaplan, J.; McCandlish, S.; Henighan, T.; Brown, T. B.; Chess, B.; Child, R.; Gray, S.; Radford, A.; Wu, J.; and Amodei, D. 2020. Scaling laws for neural language models.

Kumar, P. 2024. Large language models (LLMs): survey, technical frameworks, and future challenges. *Artificial Intelligence Review*, 57(10): 260.

Li, N.; Pan, A.; Gopal, A.; Yue, S.; Berrios, D.; Gatti, A.; Li, J. D.; Dombrowski, A.-K.; Goel, S.; Phan, L.; et al. 2024a. The wmdp benchmark: Measuring and reducing malicious use with unlearning.

Li, W.; Li, L.; Xiang, T.; Liu, X.; Deng, W.; and Garcia, N. 2024b. Can multiple-choice questions really be useful in detecting the abilities of LLMs?

Meng, K.; Bau, D.; Andonian, A.; and Belinkov, Y. 2022a. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35: 17359–17372.

Meng, K.; Sharma, A. S.; Andonian, A.; Belinkov, Y.; and Bau, D. 2022b. Mass-editing memory in a transformer. *arXiv preprint arXiv:2210.07229*.

Protection, D. 2018. General data protection regulation.

Sun, Y.; Li, X.; Dalal, K.; Xu, J.; Vikram, A.; Zhang, G.; Dubois, Y.; Chen, X.; Wang, X.; Koyejo, S.; et al. 2024. Learning to (learn at test time): Rnns with expressive hidden states.

Tandon, A.; Dalal, K.; Li, X.; Koceja, D.; Rød, M.; Buchanan, S.; Wang, X.; Leskovec, J.; Koyejo, S.; Hashimoto, T.; et al. 2025. End-to-end test-time training for long context.

Wang, K.; Zhang, G.; Zhou, Z.; Wu, J.; Yu, M.; Zhao, S.; Yin, C.; Fu, J.; Yan, Y.; Luo, H.; et al. 2025a. A comprehensive survey in llm (-agent) full stack safety: Data, training and deployment.

Wang, Q.; Zhou, J. P.; Zhou, Z.; Shin, S.; Han, B.; and Weinberger, K. Q. 2025b. Rethinking llm unlearning objectives: A gradient perspective and go beyond.

Wang, Y.; Wei, J.; Liu, C. Y.; Pang, J.; Liu, Q.; Shah, A. P.; Bao, Y.; Liu, Y.; and Wei, W. 2024. Llm unlearning via loss adjustment with only forget data.

Yao, Y.; and Xu, X. 2024. Large language model unlearning. *Advances in Neural Information Processing Systems*, 37: 105425–105475.

Yi, H.; Wang, K.; Li, Q.; Yu, M.; Lin, L.; Xi, G.; Wu, H.; Hu, X.; Li, K.; and Liu, Y. 2025. SaFeR-VLM: Toward Safety-aware Fine-grained Reasoning in Multimodal Models.

Zade, S. Z.; Zhou, X.; Liu, S.; and Zhu, D. 2026. Attention Smoothing Is All You Need For Unlearning.

Zhang, R.; Lin, L.; Bai, Y.; and Mei, S. 2024. Negative preference optimization: From catastrophic collapse to effective unlearning.

Zhang, Y.; and Lin, L. 2025. Enj: Optimizing noise with genetic algorithms to jailbreak lsms.